

SRE.ai

Your AI-Powered Site Reliability Engineer

Full Product Requirements Document

Version 1.0 • Author: Santosh Kumar • September 2026

Confidential — Product & Engineering Reference

Mission: Give every small startup the incident response capability of a world-class SRE team — without hiring one.

Table of Contents

- 1. Problem Statement
- 2. Market Opportunity
- 3. Target Customer
- 4. Product Vision & Goals
- 5. Core Features (MVP)
- 6. Full Feature Roadmap
- 7. System Architecture
- 8. Data Models
- 9. API Design
- 10. Integrations
- 11. Monetisation Strategy
- 12. Go-to-Market Plan
- 13. Success Metrics
- 14. Risks & Mitigations
- 15. Build Timeline

1. Problem Statement

The Pain

It is 3:14am. An alert fires. CPU on your auth service just hit 98%. Error rate crossed 5%. Your users are seeing 503s. You have no SRE. Your on-call engineer — probably a backend developer — gets paged, wakes up groggy, opens 5 dashboards, reads thousands of log lines, tries to form a hypothesis, makes a judgment call under pressure, and hopes they're right. This process takes 15 to 45 minutes. Every minute costs you users, revenue, and trust.

This is the reality for the majority of software companies — the ones that are not Google, not Stripe, not Razorpay. Small startups shipping real products with real users, running on AWS or GCP or a mix, with no dedicated SRE team and no budget to hire one. A senior SRE in India costs 30-50 LPA. In the US, \$150k-\$250k. Most early-stage companies cannot afford that. So they wake up engineers at 3am instead.

The Root Cause

- Incident detection is reactive — alerts fire after users are already affected
- Diagnosis is manual — a human has to correlate logs, metrics, traces, and recent deploys
- Institutional knowledge lives in people's heads — the engineer who fixed this 6 months ago may have left
- Action is slow — even after diagnosis, humans debate whether to act and risk making it worse
- Post-mortems are optional — most startups skip them, so the same incident recurs

The Status Quo Solutions — and Why They Fail

Solution	What it does	Why it fails for startups
PagerDuty	Alert routing + on-call scheduling	Just routes the page — doesn't diagnose or fix anything
Datadog	Monitoring + dashboards	Expensive (\$\$\$), requires setup expertise, still needs human to interpret
OpsGenie	Alert management	Same as PagerDuty — no intelligence
Incident.io	Incident coordination	Assumes you already have an SRE team to coordinate
Hiring an SRE	Human expertise	30-50 LPA cost, hard to hire, doesn't scale to 24/7

The gap: every existing tool assumes a human SRE exists. SRE.ai is the first tool built for companies that don't have one.

2. Market Opportunity

Market Size

Market	Size
Total global startups (active)	~150 million
Startups with production systems but no SRE	~5 million
Startups paying for monitoring tools already	~500,000
SRE.ai serviceable market (TAM)	\$2.4B+ annually
Initial target: Indian startups on AWS/GCP	~25,000 companies

Tailwinds

- AI agents are now capable of reasoning over logs and metrics — this was not possible 2 years ago
- LLM costs have dropped 90% in 18 months — running an AI agent per incident is now economically viable
- Cloud complexity is increasing — more services, more failure modes, more alert fatigue
- SRE hiring is getting harder and more expensive globally

- Observability tooling (Prometheus, Grafana, Sentry) is now standard even at very small startups — the data SRE.ai needs already exists

Key insight: The data exists. The tools exist. What's missing is the intelligence layer that connects them and acts on them. That is SRE.ai.

3. Target Customer

Primary Persona: The Overloaded Founder-Engineer

Attribute	Detail
Who	CTO or lead backend engineer at a seed to Series A startup
Team size	3-15 engineers, no dedicated SRE or DevOps
Stack	AWS or GCP, Docker, Kubernetes or ECS, PostgreSQL or MongoDB
Monitoring	Already using Prometheus + Grafana, Sentry, or Datadog lite
Pain frequency	2-5 production incidents per month, at least 1 after hours
Current solution	Engineer on-call rotation, Slack alerts, manual debugging
Willingness to pay	\$50-\$300/month for a tool that eliminates 3am pages
Decision maker	Yes — CTO buys tools directly, no procurement process

Secondary Persona: The Solo Developer with Production Users

- Building a SaaS product alone or with 1-2 others
- No on-call rotation — just them, always
- Would pay \$20-\$50/month to sleep through the night
- Entry point to the product — lower friction, faster conversion

Anti-Persona — Who SRE.ai is NOT for (yet)

- Enterprises with existing SRE teams — they need different tooling
- Non-technical founders — they can't set up integrations
- Companies without any monitoring — SRE.ai needs data to work

4. Product Vision & Goals

Vision Statement

SRE.ai is an AI agent that monitors your production systems, detects incidents before they become outages, diagnoses root causes in seconds, and resolves common issues autonomously — so your engineers sleep through the night.

Product Goals

1. Reduce mean time to resolution (MTTR) from 30+ minutes to under 5 minutes for common incident types
2. Eliminate false-positive alert fatigue — only page a human when the AI cannot resolve with high confidence
3. Build institutional memory — every incident makes the system smarter, so the same problem never takes the same time twice
4. Be stack-agnostic — work with any monitoring stack via Prometheus, Grafana, Sentry, and webhook integrations
5. Be affordable for startups — pricing anchored at what a startup would pay for one monitoring tool, not one SRE

Non-Goals for v1

- Full autonomous infrastructure provisioning (Terraform, etc.) — too risky without human oversight
- Multi-cloud cost optimisation — separate product category
- Security incident response — different buyer, different compliance requirements
- Support for on-premise infrastructure — cloud-first for v1

5. Core Features — MVP (v1.0)

Feature 1: Alert Ingestion Layer

SRE.ai connects to your existing monitoring stack via webhooks and pull integrations. No re-instrumentation required.

- Prometheus Alertmanager webhook receiver
- Grafana alert webhook receiver
- Sentry webhook for error rate spikes
- Generic webhook endpoint for any custom alerting tool
- AWS CloudWatch alarm → SNS → SRE.ai bridge
- Alert deduplication — same alert firing from multiple sources counted once
- Alert enrichment — auto-attach service metadata, recent deploys, owner info

Feature 2: Intelligent Diagnosis Engine

When an alert fires, the diagnosis engine automatically collects context and forms a root-cause hypothesis.

- Log ingestion: pull last N minutes of logs from CloudWatch, Datadog, or Loki
- Metrics snapshot: pull CPU, memory, latency, error rate from Prometheus/Grafana
- Deploy correlation: check if a deploy happened in the last 30 minutes (GitHub webhook integration)
- Dependency check: test if upstream/downstream services are healthy
- Semantic incident search: vector search over past incidents to find similar resolved events
- LLM agent diagnosis: synthesises all context into a structured root-cause hypothesis
- Citation enforcement: every claim in the diagnosis must cite a specific log line, metric, or past incident — no hallucination

Citation enforcement is the technical differentiator. Every other AI monitoring tool lets the LLM hallucinate a plausible-sounding root cause. SRE.ai only asserts what it can prove from retrieved data.

Feature 3: Confidence-Gated Action Layer

SRE.ai does not act blindly. Every potential action is scored for confidence before execution.

Confidence Level	Threshold	Action Taken
High	> 85%	Auto-execute: restart service, scale pod, clear cache, revert last deploy
Medium	60-85%	Draft a fix: open a GitHub PR or Slack draft, notify on-call for approval
Low	< 60%	Escalate: page on-call via PagerDuty/Slack with full diagnosis context packet

- All auto-executed actions are logged with full reasoning trail
- One-click rollback for any auto-executed action
- Human can override confidence threshold per service (e.g. payment service: always escalate)

Feature 4: Incident Memory & Learning

Every resolved incident is stored as a structured memory that improves future diagnosis.

- Incident stored: alert → diagnosis → action → resolution time → outcome
- Vector embedded for semantic search — future similar incidents retrieved automatically
- Resolution patterns extracted — 'when X + Y occurs on service Z, action A resolves it in 94% of cases'
- Auto-generated post-mortem in markdown — timeline, root cause, resolution, prevention
- Runbook generator — after 3+ similar incidents resolved the same way, SRE.ai drafts a runbook

Feature 5: Notification & Escalation Layer

SRE.ai only wakes humans when it needs to. When it does, it gives them everything they need instantly.

- Slack integration — all incident updates, diagnoses, and action results in a dedicated channel
- PagerDuty integration — escalation with full context attached
- Email digest — daily summary of all incidents, resolutions, and trends
- Context packet on escalation: timeline, hypothesis, confidence score, similar past incidents, suggested next steps
- Silence windows — no pages during defined hours unless P1 (user-impacting) severity

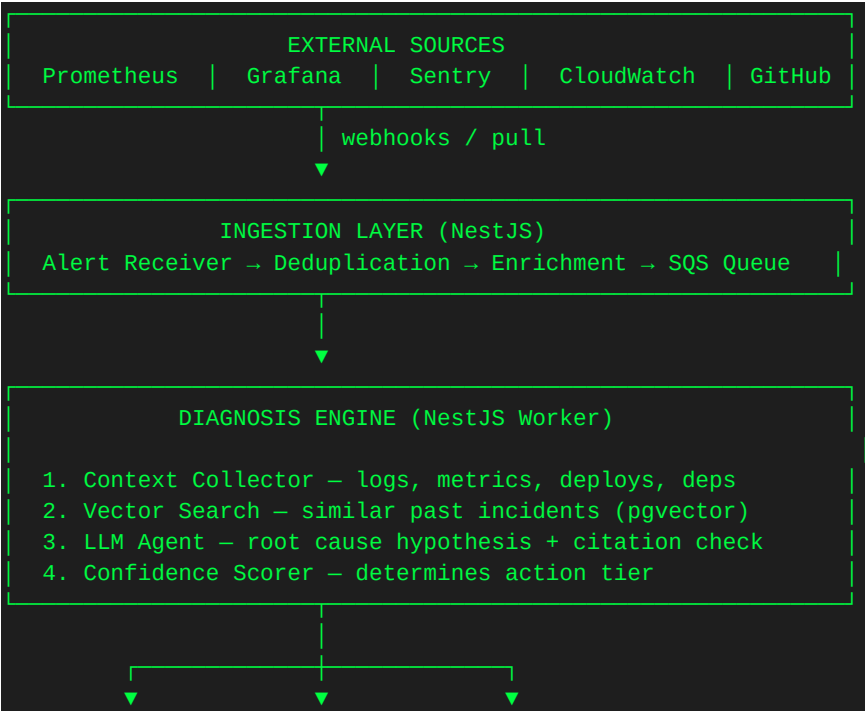
6. Full Feature Roadmap

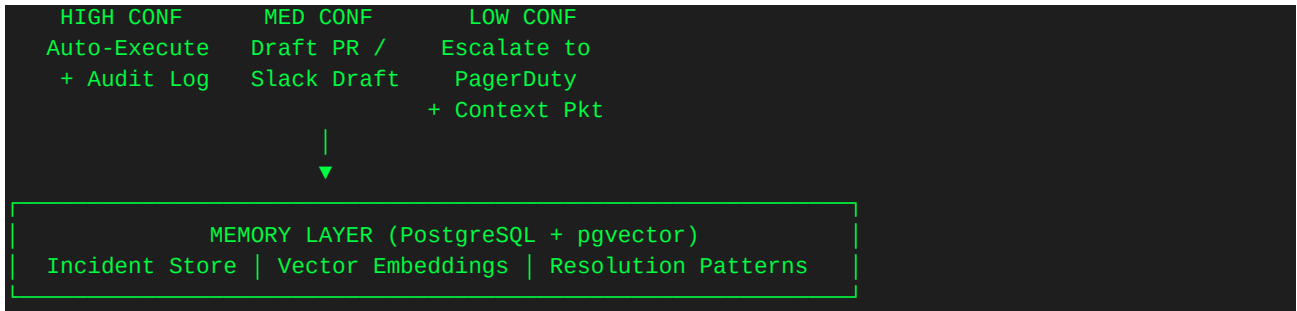
Feature	Phase	Priority	Complexity
Alert ingestion (Prometheus, Sentry, Grafana)	v1.0 MVP	P0	Medium
LLM diagnosis with citation enforcement	v1.0 MVP	P0	High
Confidence-gated action layer	v1.0 MVP	P0	High
Incident memory + vector search	v1.0 MVP	P0	High
Slack + PagerDuty notifications	v1.0 MVP	P0	Low

Auto-generated post-mortems	v1.0 MVP	P1	Medium
GitHub deploy correlation	v1.0 MVP	P1	Low
Dashboard: incident history + trends	v1.1	P1	Medium
Runbook auto-generator	v1.1	P1	Medium
Kubernetes pod management actions	v1.1	P1	High
AWS RDS / ElastiCache health checks	v1.2	P1	Medium
Multi-service dependency mapping	v1.2	P2	High
Anomaly detection (proactive, pre-alert)	v2.0	P1	High
Capacity forecasting	v2.0	P2	High
Custom runbook execution	v2.0	P1	Medium
Multi-tenant (agency use case)	v2.0	P2	High
SOC2 compliance report generation	v3.0	P2	Medium
Voice call escalation (Callease-style)	v3.0	P3	High

7. System Architecture

High-Level Architecture





Tech Stack Decision

Layer	Technology & Reason
API + Services	NestJS + TypeScript — your production stack, modular, testable
Queue	AWS SQS — you've run this at 10k msgs/day, proven reliability
Job Processing	BullMQ — async diagnosis jobs with retry and priority
Primary DB	PostgreSQL — relational incident records, audit logs
Vector Store	pgvector extension — keeps stack simple, no Pinecone cost for v1
Cache	Redis — incident state, deduplication window, session data
LLM	OpenAI GPT-4o — best reasoning for diagnosis; Claude as fallback
Embeddings	OpenAI text-embedding-3-small — cheap, fast, high quality
Infra	AWS ECS Fargate + Docker — your existing deployment pattern
Observability	Sentry (errors) + CloudWatch (infra) + Winston structured logging

Key Architectural Decisions

Why SQS over direct processing?

An alert storm (50 alerts in 30 seconds) must not overwhelm the diagnosis engine. SQS acts as a buffer. Each alert is a job. The diagnosis engine processes at its own pace with priority (P1 incidents jump the queue). This is the same pattern you used at Sthambh.

Why pgvector over Pinecone?

For v1, keeping everything in PostgreSQL dramatically simplifies ops. One database, one backup strategy, one connection pool. pgvector handles up to ~1M vectors efficiently — more than enough for years of incident history at startup scale. Migrate to Pinecone when vector query latency becomes a bottleneck.

Why citation enforcement?

The LLM diagnosis agent is constrained by a system prompt that requires every factual claim to be followed by a citation tag referencing the specific retrieved document. Before the diagnosis is surfaced to a user or used to drive an action, a secondary validation pass checks that all citations are real and present in the retrieved context. A diagnosis that fails citation validation is downgraded to Low Confidence and escalated to a human.

8. Data Models

Core Entities

```
-- Incident
CREATE TABLE incidents (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  alert_source VARCHAR(50) NOT NULL, -- 'prometheus' | 'sentry' | 'grafana'
  service_name VARCHAR(100) NOT NULL,
  severity     VARCHAR(10) NOT NULL, -- 'P1' | 'P2' | 'P3'
  title        TEXT NOT NULL,
  raw_alert     JSONB NOT NULL,
  status        VARCHAR(20) DEFAULT 'detecting',
  -- 'detecting' | 'diagnosing' | 'acting' | 'resolved' | 'escalated'
  opened_at     TIMESTAMPTZ DEFAULT NOW(),
  resolved_at   TIMESTAMPTZ,
  mtrr_seconds  INTEGER,
  embedding     vector(1536) -- pgvector: for semantic search
);
```

```
-- Diagnosis
CREATE TABLE diagnoses (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id  UUID REFERENCES incidents(id),
  hypothesis   TEXT NOT NULL,        -- LLM output
  citations    JSONB NOT NULL,       -- [{claim, source, line}]
  confidence   DECIMAL(4,3),        -- 0.000 to 1.000
  context_used JSONB,               -- logs, metrics, past incidents
  similar_incidents UUID[],         -- top 3 semantic matches
  created_at   TIMESTAMPTZ DEFAULT NOW()
);
```

```
-- Action
CREATE TABLE actions (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  incident_id  UUID REFERENCES incidents(id),
  action_type  VARCHAR(50), -- 'auto_execute' | 'draft_pr' | 'escalate'
  action_taken TEXT,        -- human-readable description
  payload      JSONB,       -- raw action parameters
  outcome      VARCHAR(20), -- 'success' | 'failed' | 'rolled_back'
  executed_at  TIMESTAMPTZ DEFAULT NOW(),
  rolled_back_at TIMESTAMPTZ
);
```

```
-- Integration (per tenant)
CREATE TABLE integrations (
  id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id   UUID NOT NULL,
  type        VARCHAR(50), -- 'prometheus' | 'sentry' | 'slack' | ...
  config      JSONB,       -- encrypted credentials + endpoints
  active      BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT NOW()
);
```

9. API Design

Core REST Endpoints

Endpoint	Description
POST /webhooks/:source	Receive alerts from Prometheus, Grafana, Sentry, etc.
GET /incidents	List all incidents with filters (status, severity, service, date)
GET /incidents/:id	Full incident detail — alert, diagnosis, actions, timeline
GET /incidents/:id/diagnosis	LLM diagnosis with citations and confidence score
POST /incidents/:id/actions	Manually trigger an action on an incident
POST /incidents/:id/resolve	Mark incident resolved with resolution note
POST /incidents/:id/rollback	Roll back last auto-executed action
GET /services	List monitored services with health status
GET /runbooks	Auto-generated runbooks from resolved incident patterns
GET /analytics/mttr	MTTR trends over time by service
POST /integrations	Connect a new monitoring tool or notification channel
GET /integrations	List active integrations and their status

Webhook Payload — Prometheus Example

```
{
  'version': '4',
  'status': 'firing',
  'alerts': [{
    'labels': { 'alertname': 'HighCPU', 'service': 'auth-service' },
    'annotations': { 'summary': 'CPU > 90% for 5 minutes' },
    'startsAt': '2026-09-15T03:14:00Z'
  }]
}
```

10. Integrations

v1.0 — Must Have

Integration	Type	What SRE.ai uses it for
Prometheus + Alertmanager	Alert source	Primary alert ingestion via webhook
Grafana	Alert source + metrics	Alert ingestion + metric snapshots for diagnosis
Sentry	Error tracking	Error rate spikes, error context for diagnosis

AWS CloudWatch	Alert + logs	CloudWatch alarms + log pulling for diagnosis
Slack	Notification	Incident updates, diagnoses, action results
GitHub	Deploy tracking	Recent commits and deploys correlated with incidents
PagerDuty	Escalation	Human escalation with context packet

v1.1 — High Value

- Datadog — alert source + metric snapshots
- Linear / Jira — auto-create incident tickets
- AWS ECS / Kubernetes — service restart, scale, rollback actions
- Loki — log pulling for GCP/self-hosted stacks

v2.0 — Strategic

- Terraform Cloud — infrastructure change correlation
- LaunchDarkly — feature flag correlation with incidents
- Twilio / Callee-style — voice escalation for critical incidents

11. Monetisation Strategy

Pricing Model: Seat-Free, Usage-Based + Plan Tiers

Startups hate per-seat pricing. SRE.ai charges by incidents monitored and actions taken — not by number of engineers.

Plan	Price	Limits & Features
Hobby (Free)	\$0/month	Up to 20 incidents/month, 1 integration, Slack only, no auto-execute
Starter	\$49/month	Up to 200 incidents/month, 5 integrations, auto-execute, email digest
Growth	\$149/month	Unlimited incidents, 15 integrations, all actions, runbooks, analytics
Pro	\$399/month	Multi-service, priority support, custom confidence thresholds, SSO
Enterprise	Custom	Multi-tenant, SLA, dedicated support, on-prem option, SOC2

Revenue Projections — Conservative

Milestone	Customers	MRR
Month 6 (launch)	20 customers (Starter avg)	~\$1,000 MRR

Month 12	100 customers (mix)	~\$8,000 MRR
Month 18	300 customers	~\$30,000 MRR
Month 24	800 customers	~\$80,000 MRR (\$1M ARR)

Additional Revenue Streams

- Usage overage — incidents above plan limit billed at \$0.10 per incident
- Action credits — auto-execute actions above plan limit billed at \$0.50 per action
- Professional services — custom integration setup for Enterprise (\$500-\$2000 one-time)
- API access — sell SRE.ai diagnosis API to other devtools companies

Path to \$1M ARR: 800 customers at \$100 average MRR. This is achievable within 24 months with focused GTM on Indian and SEA startup ecosystems where SRE hiring is expensive and developer tooling adoption is high.

12. Go-to-Market Plan

Phase 1 — Private Beta (Month 1-3)

- Recruit 10 design partners from your personal network — startup founders and CTOs
- Free access in exchange for weekly feedback calls
- Instrument everything — every action, every diagnosis, every false positive
- Goal: reduce MTTR by 50%+ for at least 7 of 10 design partners

Phase 2 — Public Launch (Month 4-6)

- Product Hunt launch — 'Your AI SRE that never sleeps'
- Hacker News Show HN post — technical deep dive on citation enforcement
- Dev.to / Hashnode article: 'How I built an AI agent that fixed my 3am production incidents'
- Twitter/X thread: show a real incident being diagnosed and resolved in <2 minutes
- Target: 500 signups in first week, 50 paid conversions in first month

Phase 3 — Growth (Month 7-18)

- Integration marketplace listings — appear in Sentry, Grafana, PagerDuty app directories
- Content marketing: weekly incident post-mortem teardowns (anonymised)
- Community: 'SRE for Startups' Slack/Discord — position SRE.ai as the sponsor
- Partnerships: Indian startup ecosystems — Razorpay DevRel, AWS Activate, YC network
- Referral program: 1 month free for every paying customer referred

GTm Channels — Priority Order

Channel	Why it works for SRE.ai
Developer content (blog + X)	Engineers discover tools through technical content, not ads
Integration marketplaces	Appear where engineers already are (Sentry, Grafana)

Startup communities	Your target customer is concentrated in YC, IndiaQuotient, Accel networks
Product Hunt / HN	One-time spike that seeds organic growth if product is strong
Founder outreach (personal)	You have direct access to startup founders — use it

13. Success Metrics

Product Metrics

Metric	Target
MTTR reduction (vs baseline)	> 60% reduction for users
Auto-resolve rate	> 40% of incidents resolved without human
Diagnosis accuracy	> 80% user-confirmed correct root cause
False positive escalation rate	< 15% of escalations were unnecessary
Citation pass rate	> 95% of diagnoses pass citation validation
Time to first value	< 30 minutes from signup to first incident processed

Business Metrics

Metric	Target (Month 12)
MRR	\$8,000+
Paying customers	100+
Monthly churn	< 5%
NPS	> 50
Avg incidents processed/customer/month	> 30

14. Risks & Mitigations

Risk	Severity	Mitigation
LLM hallucinates a wrong diagnosis → auto-execute makes it worse	Critical	Citation enforcement + conservative default confidence thresholds + one-click rollback
OpenAI API outage during critical incident	High	Claude as fallback model; local diagnosis cache for common patterns
Customer doesn't trust auto-execute	High	Default to draft-and-approve mode; auto-execute is opt-in per service

Too many false positives from Prometheus	Medium	Alert deduplication + noise filter + per-service silence windows
LLM cost becomes uneconomical at scale	Medium	Fine-tune smaller model on incident corpus; cache common diagnosis patterns
Competitor (Datadog AI) ships similar feature	Medium	Speed of execution + startup focus + pricing — Datadog won't go cheap
Customers churn after first month (set-and-forget)	Medium	Weekly incident digest, trend reports, runbook updates — stay visible

15. Build Timeline

Solo Builder Estimate (You + Claude Code)

Phase	Duration	Deliverable
Phase 0: Foundation	Week 1-2	NestJS monorepo, PostgreSQL + pgvector setup, auth, tenant model
Phase 1: Ingestion	Week 3-4	Prometheus + Sentry webhook receivers, SQS queue, deduplication
Phase 2: Diagnosis	Week 5-7	Context collector, vector search, LLM agent, citation enforcement
Phase 3: Actions	Week 8-9	Confidence scorer, Slack notifications, PagerDuty escalation
Phase 4: Memory	Week 10-11	Incident store, post-mortem generator, resolution patterns
Phase 5: Dashboard	Week 12-13	Next.js dashboard, incident timeline, analytics
Phase 6: Beta	Week 14-16	10 design partners, feedback, bug fixes, performance tuning
Public Launch	Month 4	Product Hunt, HN, content push

Realistic pace: 2-3 hours per day on this alongside your job. At that pace, 16 weeks is achievable. Phase 2 (diagnosis engine) is the hardest — budget extra time there.

This is not a resume project.

This is a real product solving a real problem.

Build it like one.